

A decorative graphic in the top-left corner consisting of a cluster of black and white circles of varying sizes, some solid and some outlined, arranged in a roughly circular pattern.

RESTA UM

PARTE III

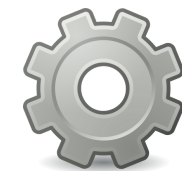
Anderson Akira Mekaru
Daniel Tsuyoshi Izumi
Marcelo Alfonso de Souza
Victor Oliveira da Nóbrega Santana

SPRINTS - CONTEXTUALIZAÇÃO

1	Apresentação do plano (Restaurar)	APRES. DO...	
2	Planejamento do jogo	SPRINT 0	✓
3	Tabuleiro inicial + seleção	SPRINT 1	✓
4	Movimentação e remoção	SPRINT 2	✓
5	Validação de regras	SPRINT 3	✓
6	Controle de jogo	SPRINT 4	⚙
7	Mensagens do jogo	SPRINT 5	⚙
8	Testes e correções	SPRINT 6	⚙
9	Deploy + monitoração	SPRINT 7	⚙
10	Documentação	SPRINT 8	📋
11	Revisão geral e ajustes	SPRINT 9	📋
12	Preparação da apresentação	SPRINT 10	📋
13	Entrega final e apresentação	ENTREGA FIN...	



Sprints realizadas anteriormente

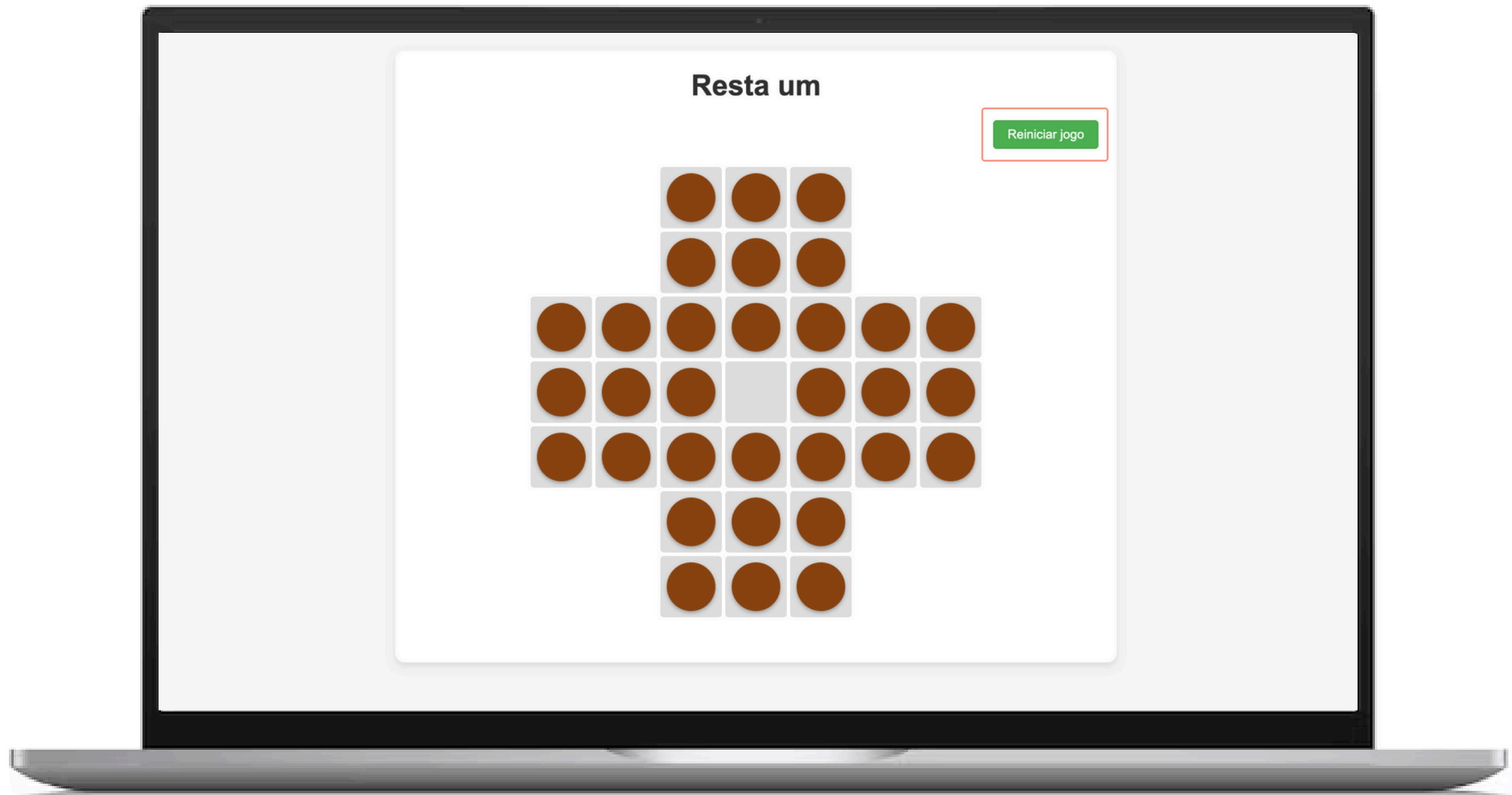


Sprints desenvolvidas



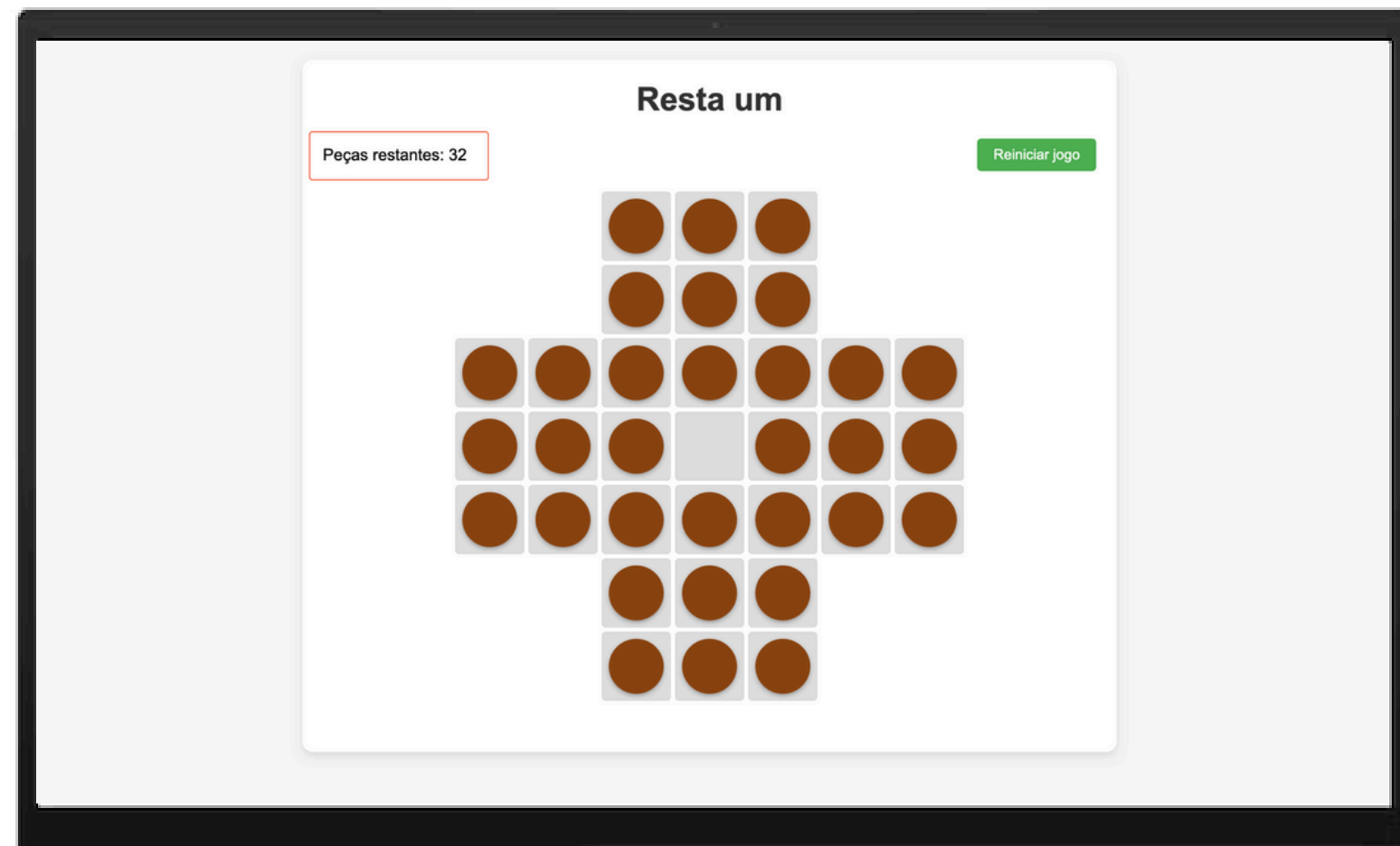
Preparação de documentos/apresentação

UI ACTION

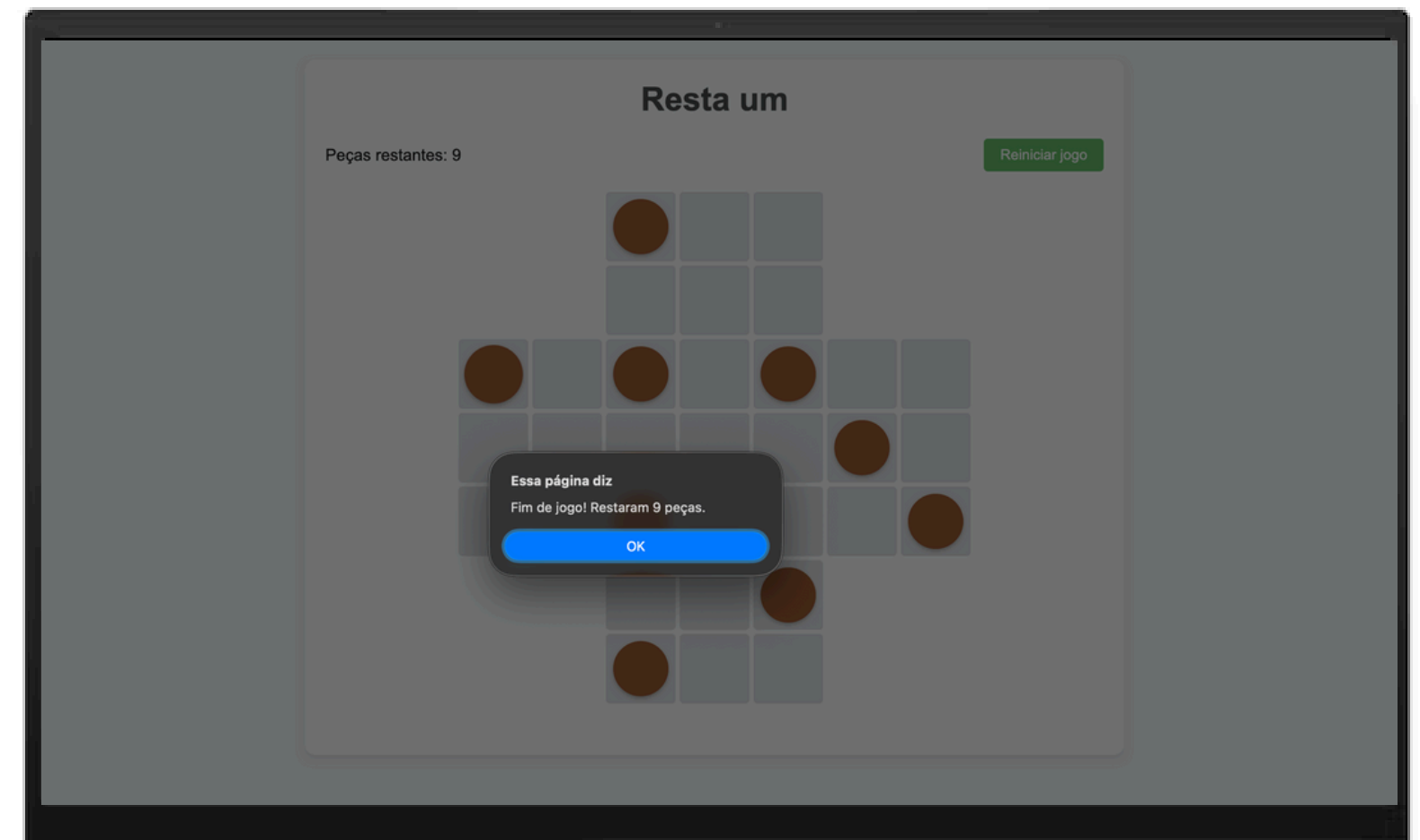


Adicionar botão para reinício do jogo

MENSAGENS INFORMATIVAS



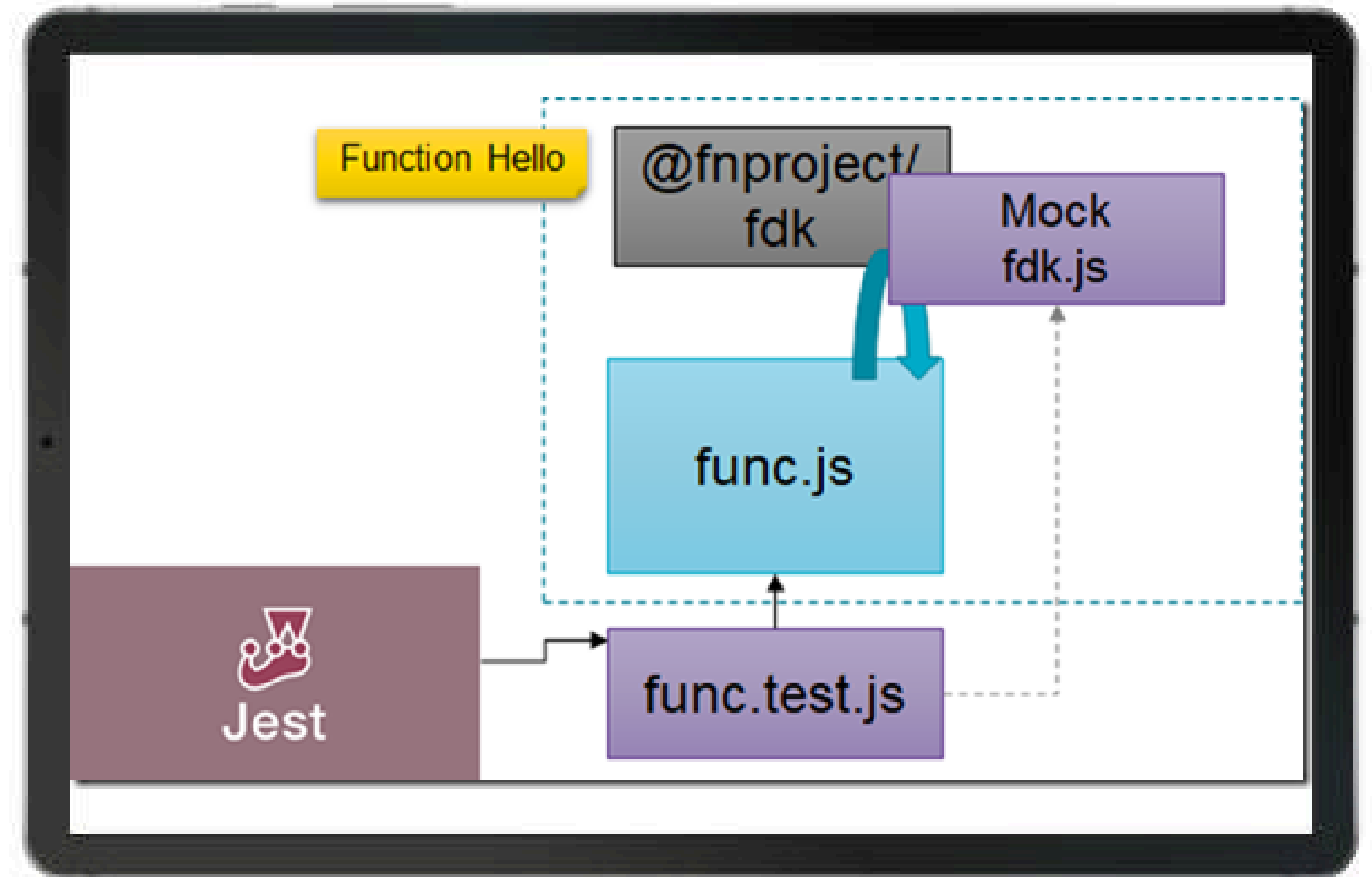
Exibir o número de peças ainda no tabuleiro



Mostrar mensagem de vitória quando sobrar apenas uma peça, ou de derrota quando não houver mais movimentos possíveis

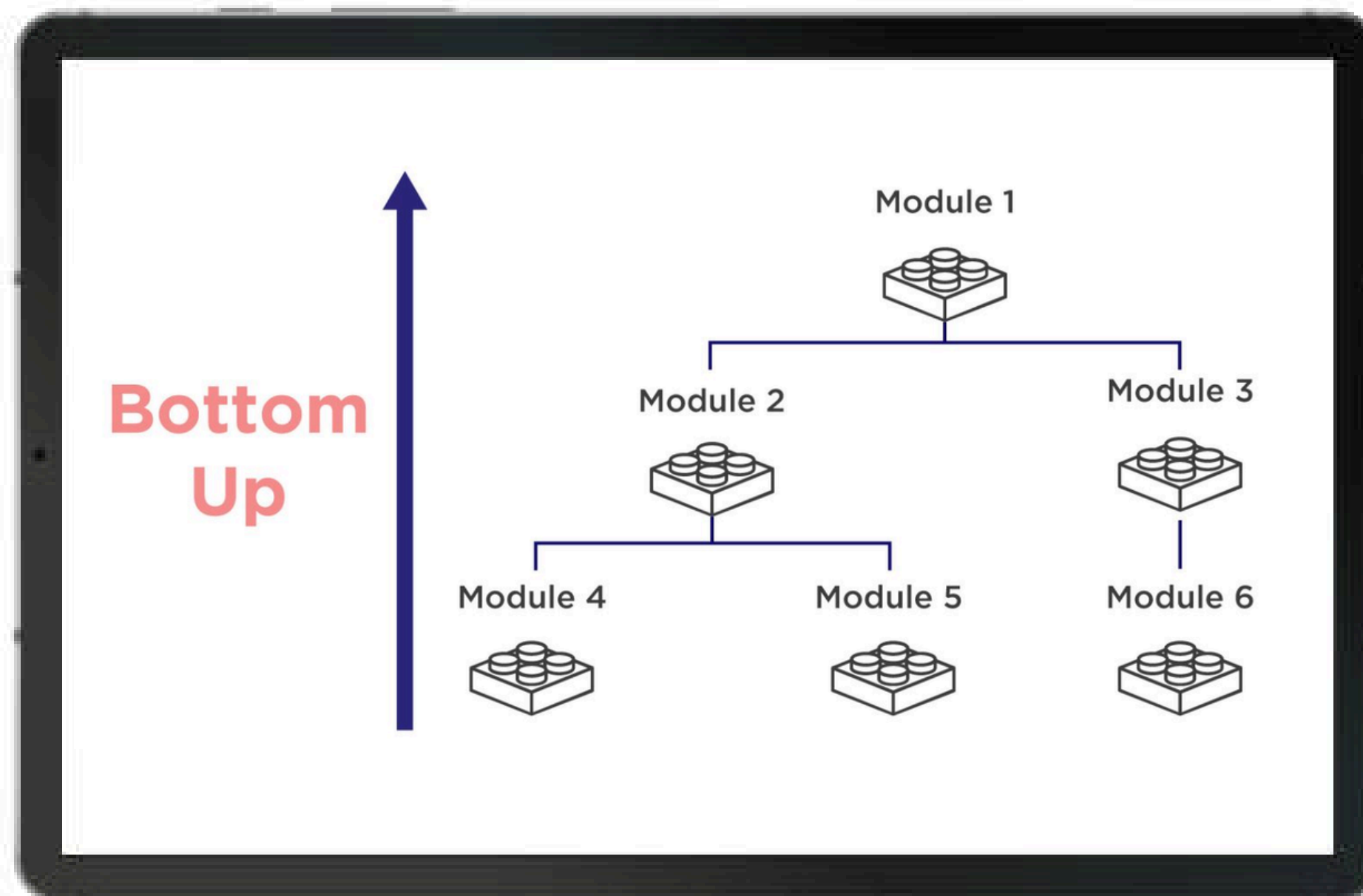
FRAMEWORK PARA TESTES

- Mantido pela Meta, o Jest é um framework de testes open-source usado para testes de unidade e integração em aplicações JavaScript/TypeScript;
- Sua configuração é simples e a execução é bastante rápida;
- Possui watch mode e gera relatórios automáticos;
- Permite simular funções, módulos e APIs.



INTEGRAÇÃO BOTTOM-UP

- Abordagem através da qual as funções mais simples e independentes são testadas primeiro;
- Uso de drivers para acionar módulos superiores ainda não implementados ou incompletos;
- A interface é conectada apenas após a estabilidade dos elementos básicos;
- Adequado ao projeto porque o jogo depende de uma lógica consistente, garantindo que a complexidade cresça de forma controlada e com menor propagação de erros.




```
1 test('movePiece() - Deve atualizar tabuleiro após movimento', async () => {
2   await delay(200);
3   expect(api.gameState.board[3][1]).toBe(1);
4   expect(api.gameState.board[3][2]).toBe(1);
5   expect(api.gameState.board[3][3]).toBe(2);
6   api.movePiece(3, 1, 3, 3);
7   await delay(200);
8   expect(api.gameState.board[3][1]).toBe(2);
9   expect(api.gameState.board[3][2]).toBe(2);
10  expect(api.gameState.board[3][3]).toBe(1);
11 });
```

```
Test Suites: 1 failed, 1 total
Tests:       3 failed, 101 passed, 104 total
Snapshots:   0 total
Time:        6.963 s, estimated 25 s
Ran all test suites.
```

Exemplo de caso de teste: ao movimentar uma peça de uma posição para outra, o estado do tabuleiro precisa ser atualizado

Resultado da execução dos testes de unidade

TESTES DE INTEGRAÇÃO

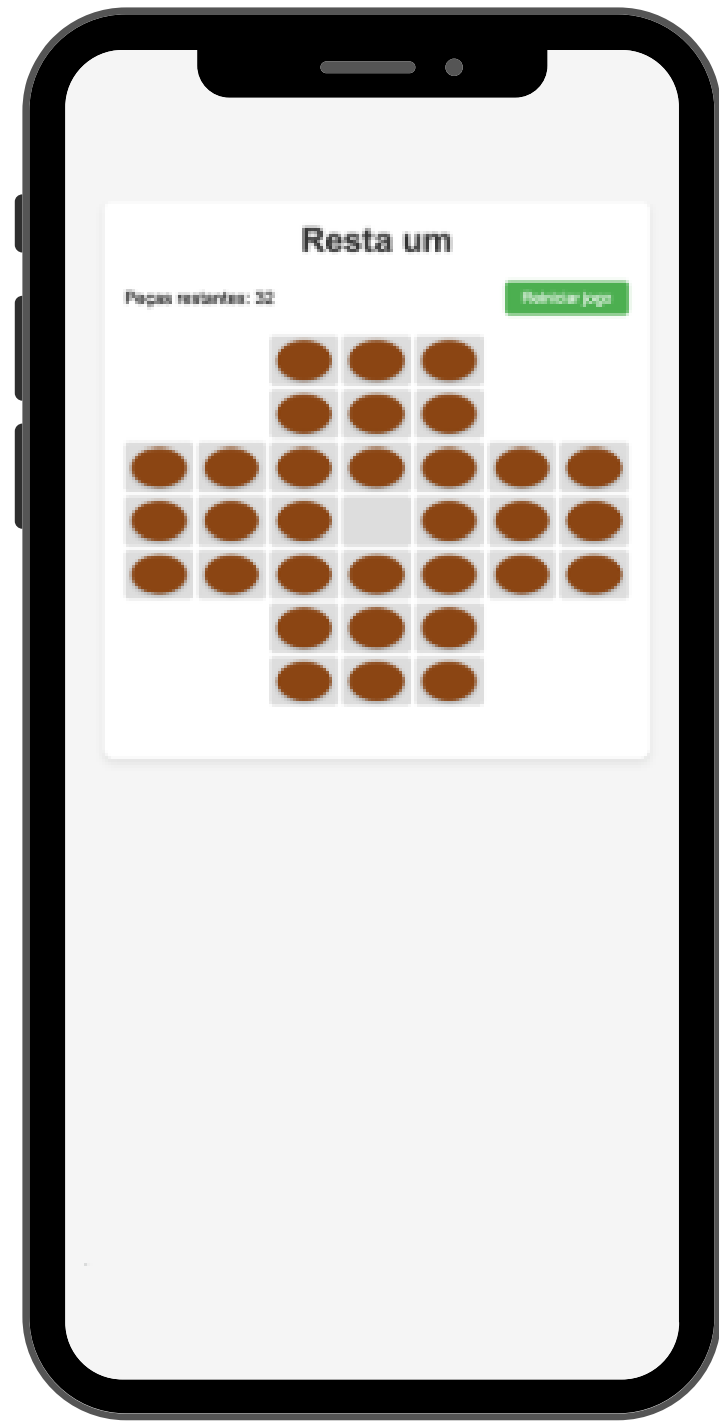
```
1 test('Debe detectar condição de vitória com 1 peça restante', async () => {
2   api.initGame();
3   await delay(600);
4
5   api.gameState.board = [
6     [0, 0, 2, 2, 2, 0, 0],
7     [0, 0, 2, 1, 2, 0, 0],
8     [2, 2, 2, 1, 2, 2, 2],
9     [2, 2, 2, 2, 2, 2, 2],
10    [2, 2, 2, 2, 2, 2, 2],
11    [0, 0, 2, 2, 2, 0, 0],
12    [0, 0, 2, 2, 2, 0, 0],
13  ];
14  api.gameState.remainingPieces = 2;
15
16  for (let row = 0; row < 7; row++) {
17    for (let col = 0; col < 7; col++) {
18      if (api.gameState.board[row][col] !== 0) {
19        api.removePieceFromCell(row, col);
20      }
21    }
22  }
23
24  api.addPieceToCell(1, 3);
25  api.addPieceToCell(2, 3);
26
27  expect(api.hasValidMoves()).toBe(true);
28
29  api.handleCellClick(1, 3);
30  await delay(20);
31  api.handleCellClick(3, 3);
32  await delay(400);
33
34  expect(api.gameState.remainingPieces).toBe(1);
35  expect(api.hasValidMoves()).toBe(false);
36 });
```

Exemplo de caso de teste: ao fazer o último movimento e sobrar apenas uma peça, a vitória é detectada.

Test Suites: 1 failed, 1 total
Tests: 1 failed, 26 passed, 27 total
Snapshots: 0 total
Time: 17.957 s
Ran all test suites.

TESTES DE ACEITAÇÃO

ALFA



- Coleta de opiniões de amigos e familiares sobre a versão inicial (local) do jogo;
- "Simples, mas funciona";
- "A interface fica muito esticada no celular";
- "O jogo não termina sozinho, preciso clicar no botão de reiniciar".

FALHAS DETECTADAS

● Testes unitários > checkValidMove() - Deve rejeitar origem com coluna negativa [3,-1]

```
expect(received).toBe(expected) // Object.is equality
```

Expected: **false**

Received: **true**

```
261 |  
262 | test('checkValidMove() - Deve rejeitar origem com coluna negativa [3,-1]', () => {  
> 263 |   expect(api.checkValidMove(3, -1, 3, 1)).toBe(false);  
    |                                     ^  
264 | });
```

● Testes de integração > Deve encerrar o jogo automaticamente quando não há mais movimentos possíveis

```
expect(received).toBe(expected) // Object.is equality
```

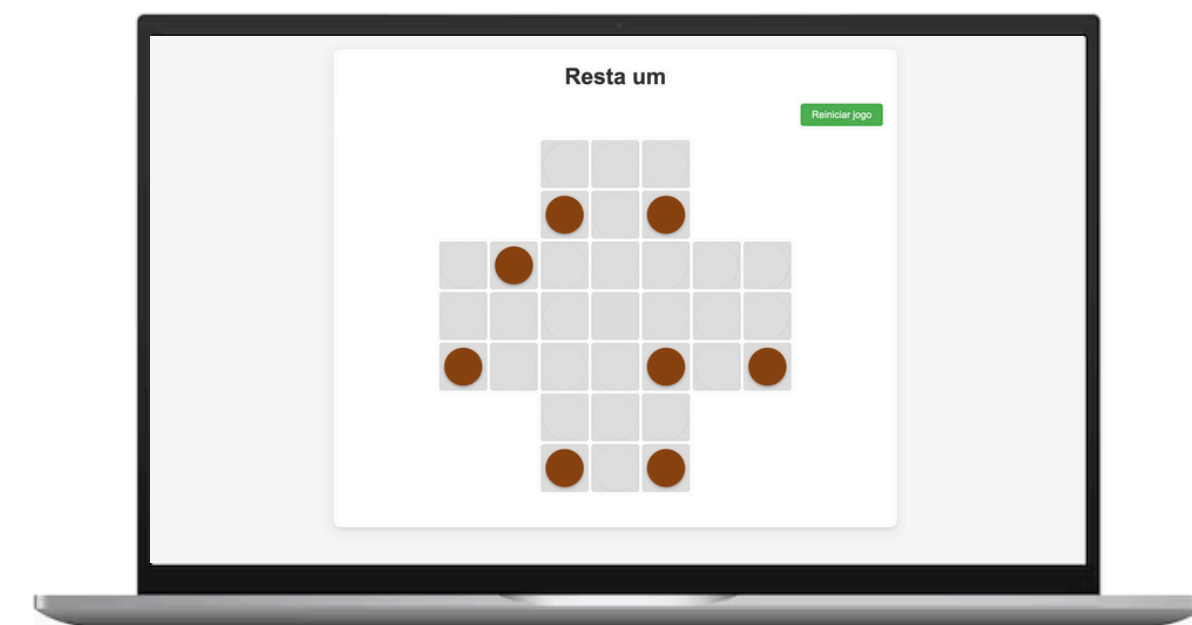
Expected: **false**

Received: **true**

```
1160 |   expect(api.gameState.remainingPieces).toBe(1);  
1161 |   expect(api.hasValidMoves()).toBe(false);  
> 1162 |   expect(api.gameState.isGameActive).toBe(false);  
    |                                     ^  
1163 | });
```

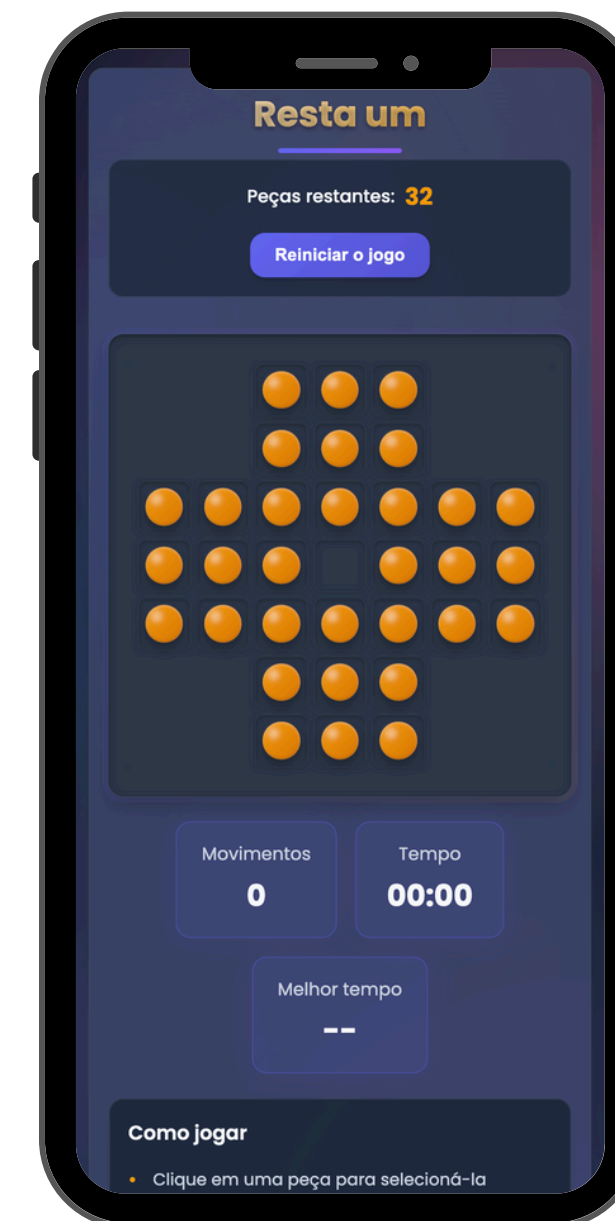
A função `checkValidMove()` permitia que uma peça fosse movida de/para uma posição inválida no tabuleiro; ela apenas verificava se a distância entre as posições era de duas casas.

O jogo continuava rolando mesmo não havendo mais movimentos possíveis.



CORREÇÕES APLICADAS

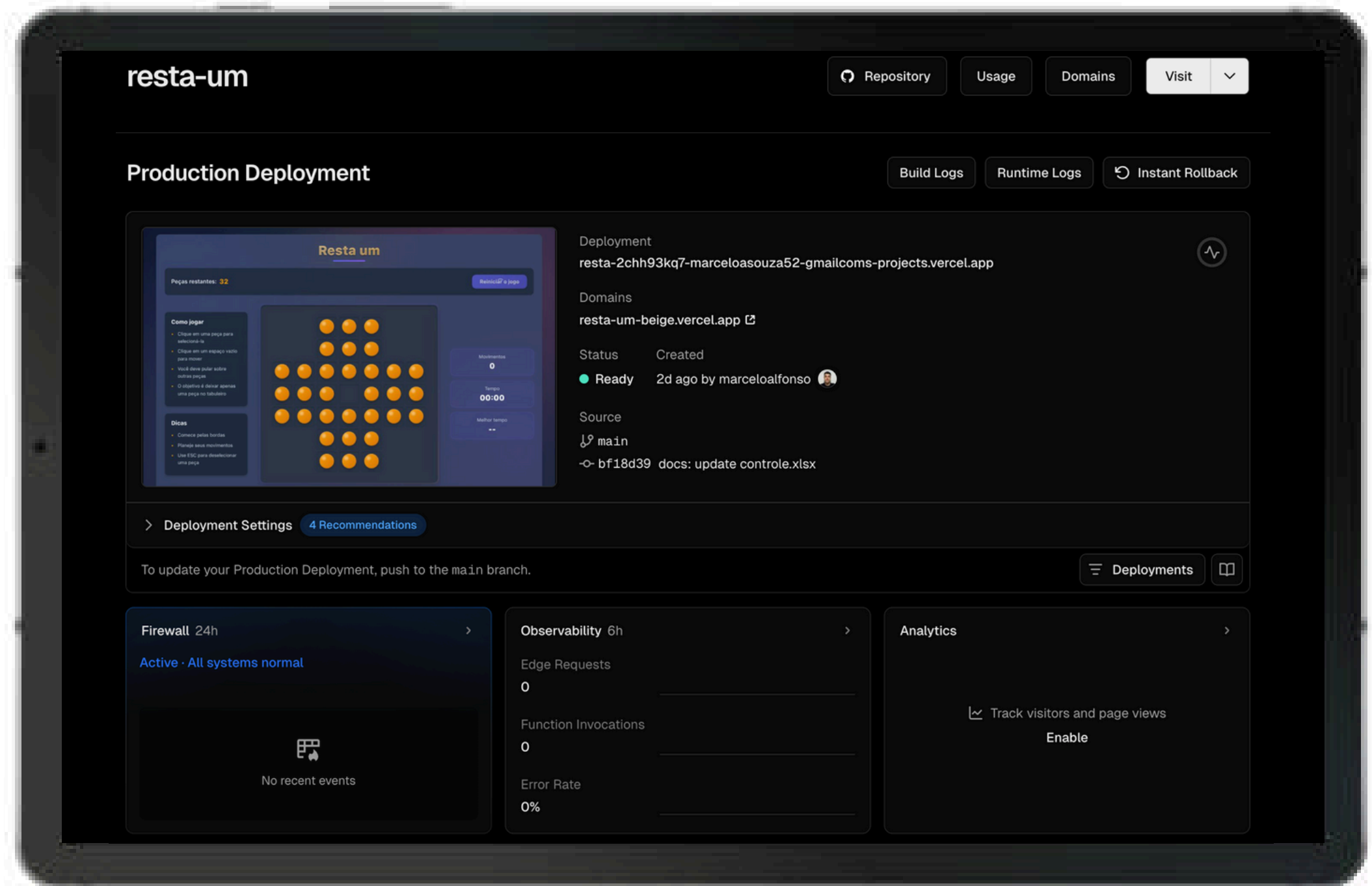
- Função `checkValidMove()` passa a verificar se as posições de origem e destino são válidas;
- Função `endGame()` é chamada quando não existem mais jogadas a serem feitas, apresentando uma mensagem de vitória ou derrota;
- O jogo tornou-se mais responsivo em dispositivos com telas menores;
- A interface foi aprimorada para melhorar a experiência do usuário (visual mais agradável, instruções de como jogar, dicas e contagem de movimentos/tempo).



```
Test Suites: 1 passed, 1 total
Tests:      168 passed, 168 total
Snapshots:  0 total
Time:       52.954 s, estimated 54 s
Ran all test suites.
```

DEPLOY DA APLICAÇÃO

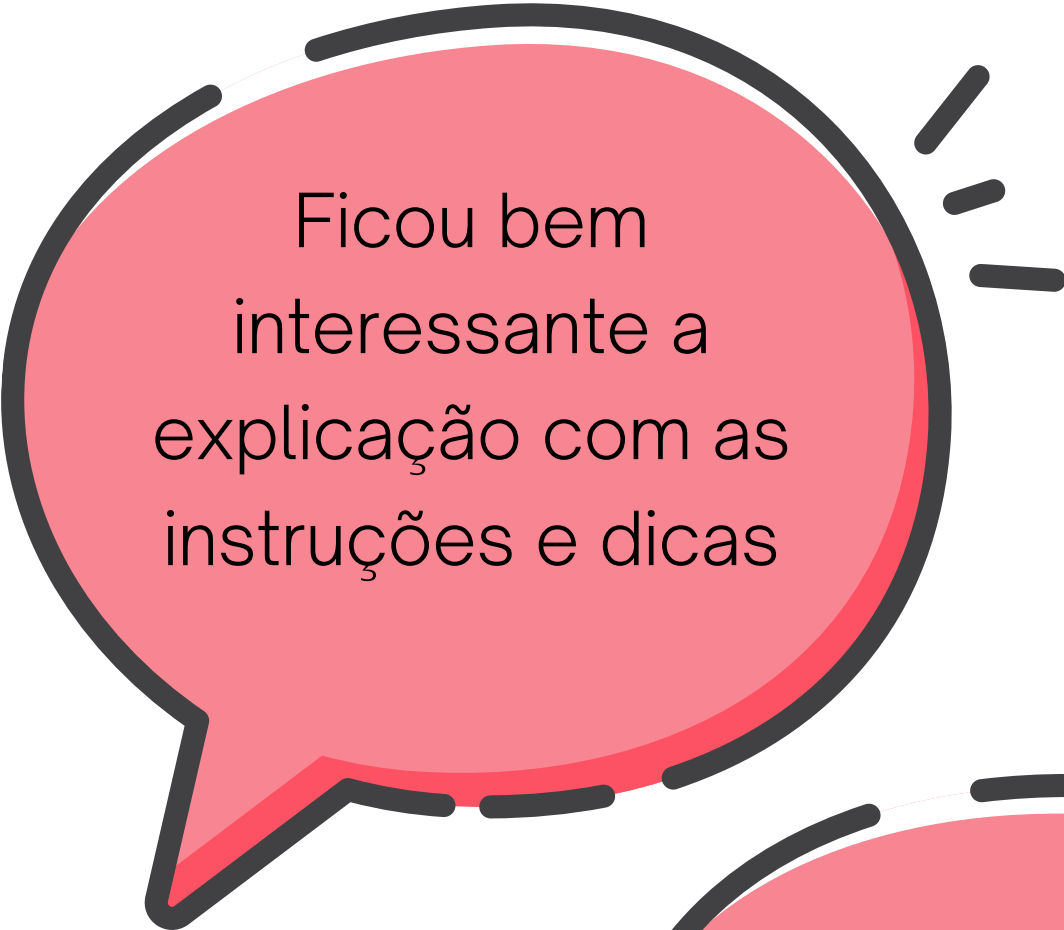
- Projeto na plataforma da Vercel;
- Detecção automática das linguagens empregadas;
- Auto deploy;
- Importação do repositório do GitHub;
- Configuração do diretório principal (src).



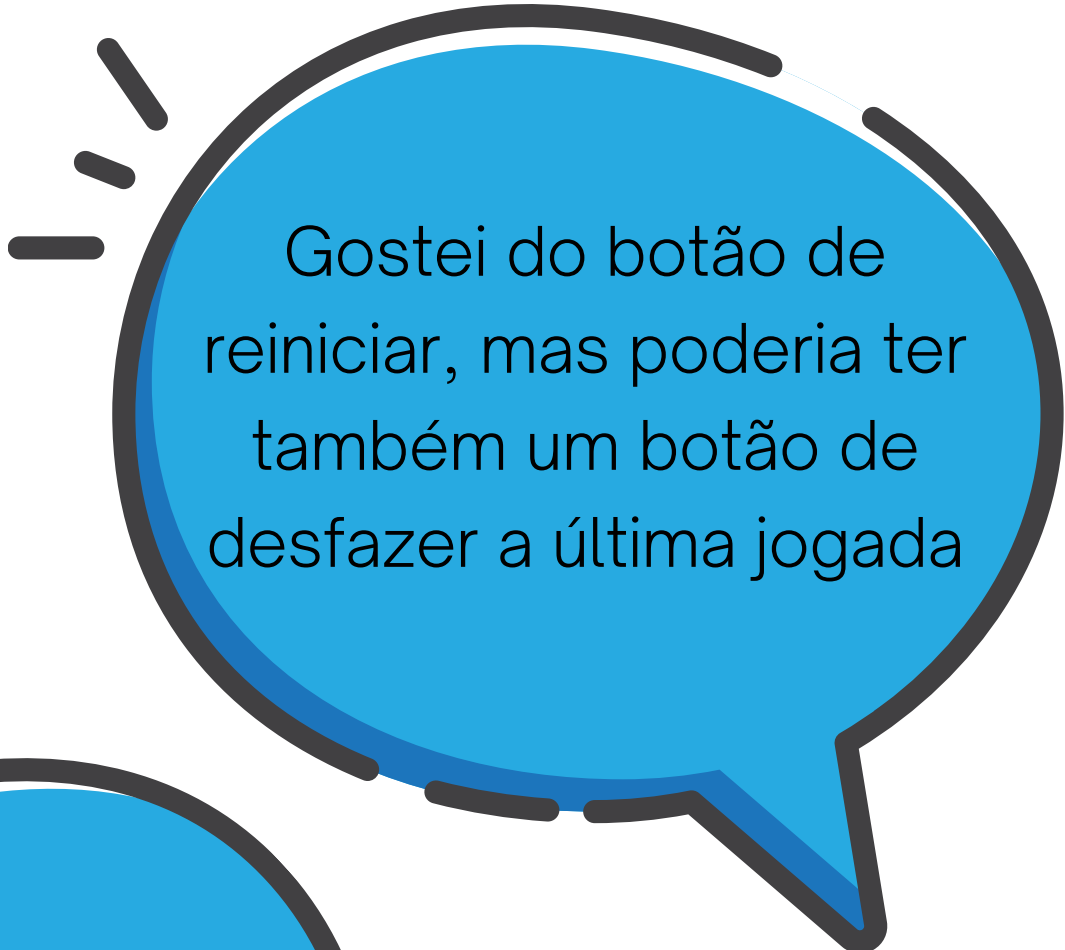
<https://resta-um-beige.vercel.app/>

TESTES DE ACEITAÇÃO

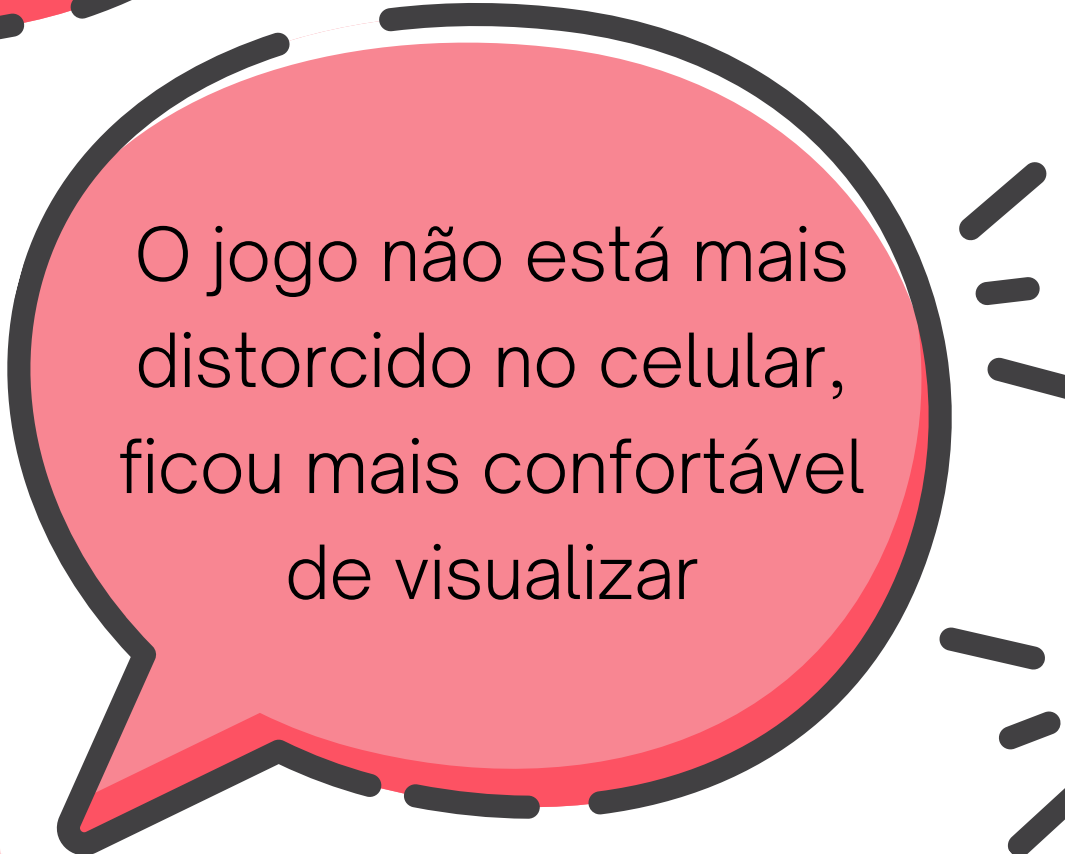
BETA

A pink speech bubble with a black outline and three short black lines radiating from the top right corner.

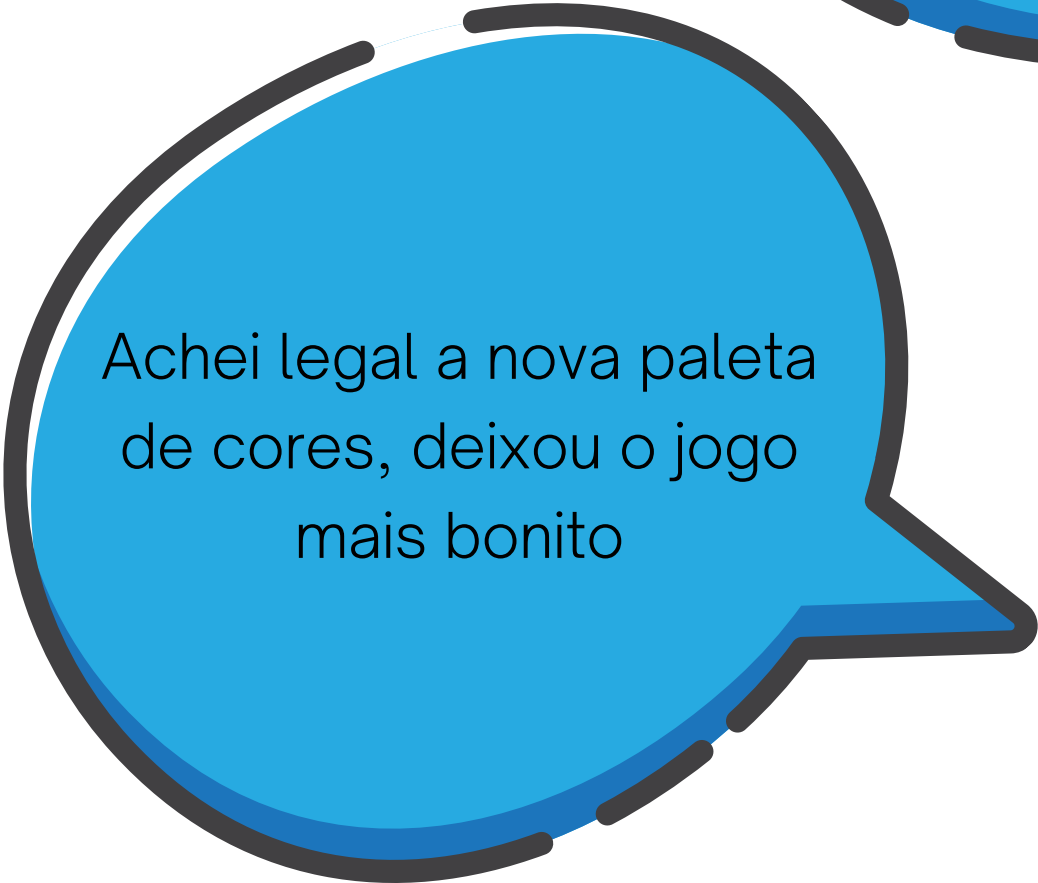
Ficou bem interessante a explicação com as instruções e dicas

A blue speech bubble with a black outline and three short black lines radiating from the top left corner.

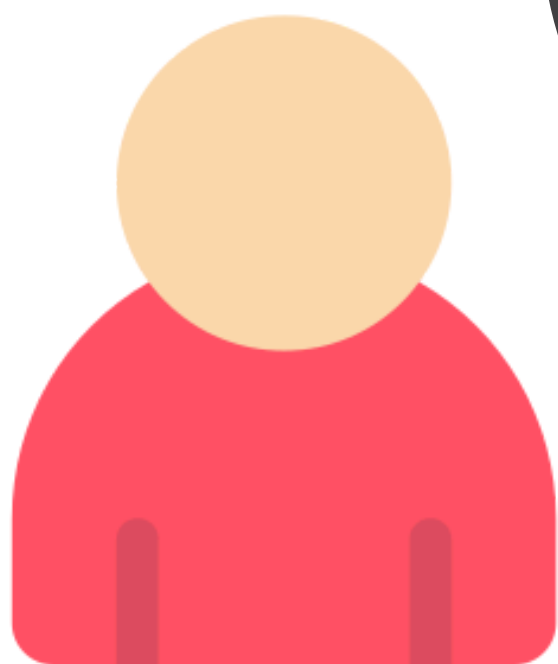
Gostei do botão de reiniciar, mas poderia ter também um botão de desfazer a última jogada

A pink speech bubble with a black outline and three short black lines radiating from the top right corner.

O jogo não está mais distorcido no celular, ficou mais confortável de visualizar

A blue speech bubble with a black outline and three short black lines radiating from the top left corner.

Achei legal a nova paleta de cores, deixou o jogo mais bonito

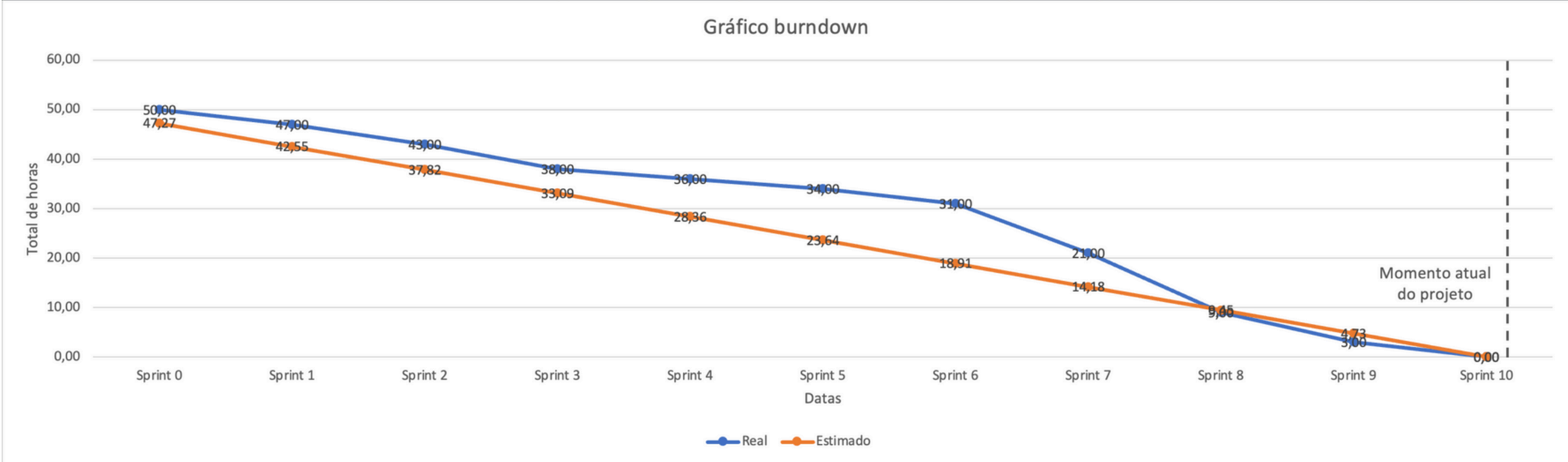


CRONOGRAMA E BURNDOWN



Atividade	Total de horas	Datas previstas											
		Sprint 0	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Sprint 6	Sprint 7	Sprint 8	Sprint 9	Sprint 10	
Definir linguagem de programação	1	1											
Escolher tecnologias (versionamento, hospedagem)	1	1											
Construir tabuleiro com peças	2		2										
Implementar seleção de peça	1		1										
Implementar movimentação de peça	2			2									
Implementar remoção de peça	2			2									
Fazer validação dos movimentos	5				5								
Construir botão para reiniciar o jogo	2					2							
Desenvolver exibição de feedback para o usuário	2						2						
Executar testes	15							3	5	5	2		
Aplicar correções	6							0	2	4			
Fazer deploy da aplicação	2								2				
Monitorar desempenho da aplicação	1								1				
Garantir documentação suficiente	3									3			
Revisar e ajustar pontos do projeto	4										4		
Preparar apresentação final	3											3	

Restante	52,00	50,00	47,00	43,00	38,00	36,00	34,00	31,00	21,00	9,00	3,00	0,00
Estimado	52,00	47,27	42,55	37,82	33,09	28,36	23,64	18,91	14,18	9,45	4,73	0,00

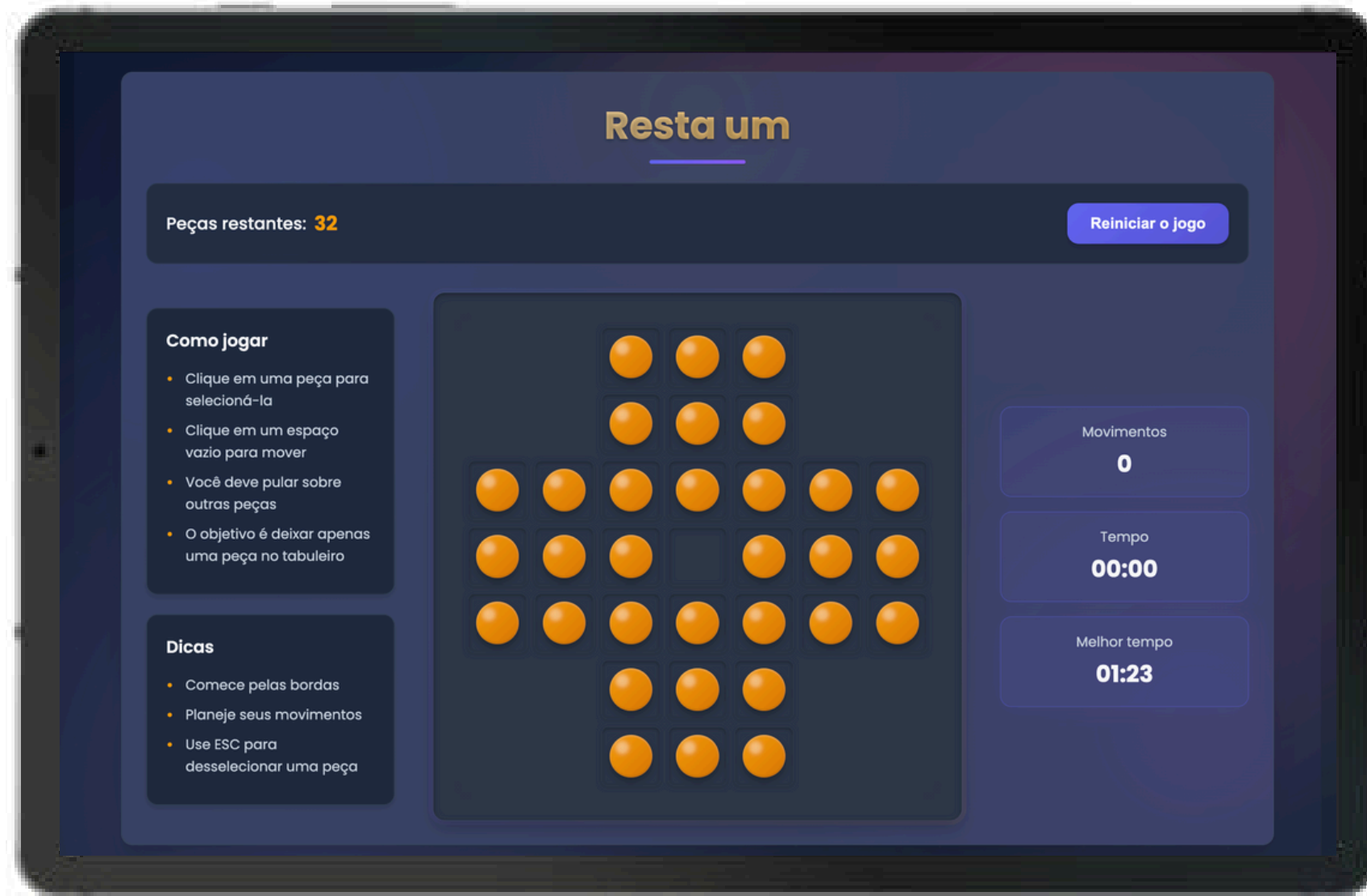


FLUXO DE TRABALHO

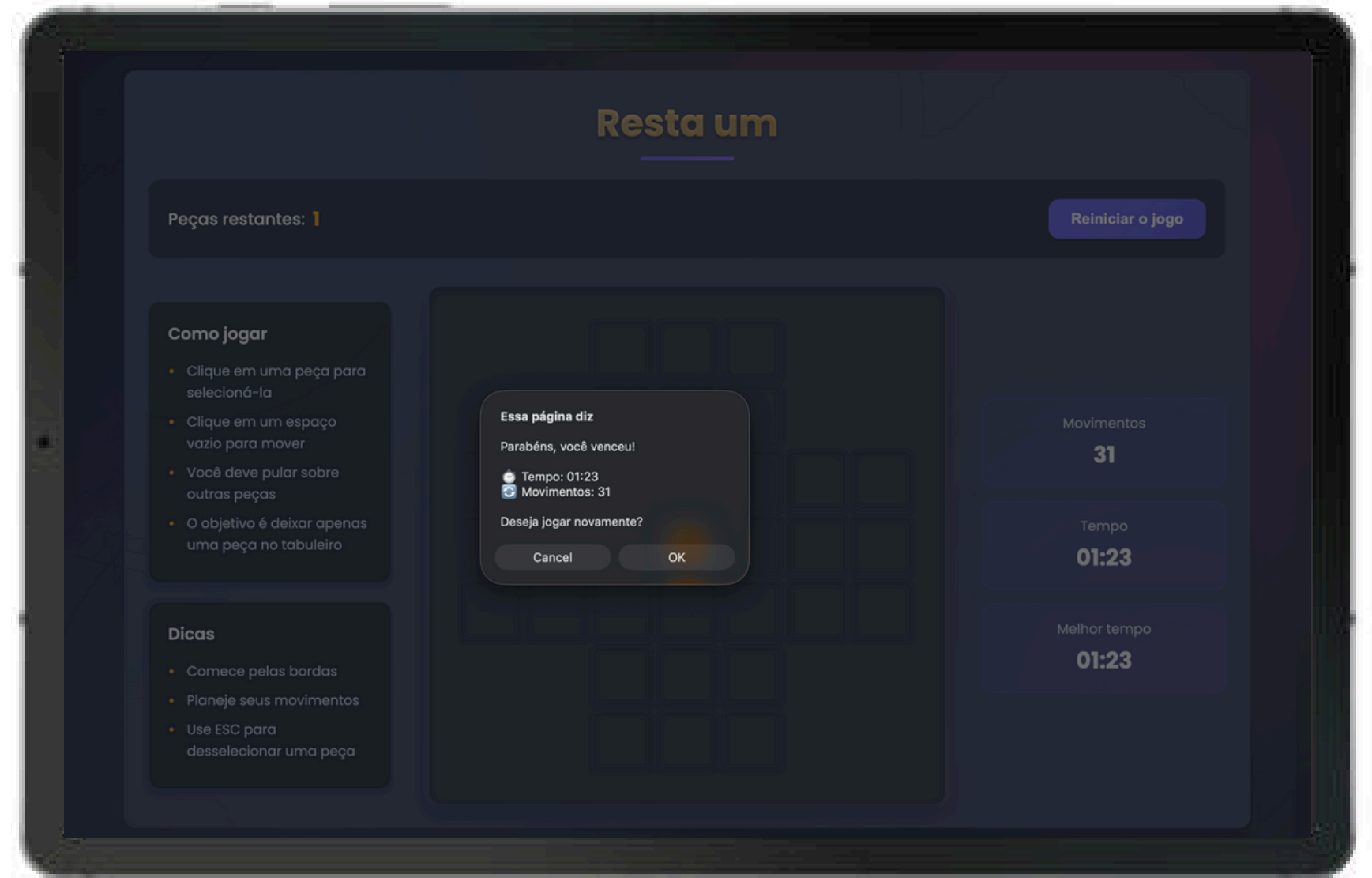
ATIVIDADE	RESPONSÁVEL	EM ANDAMENTO?	CONCLUÍDA?	OBSERVAÇÕES
Definir linguagem de programação	Todos		Sim	
Escolher tecnologias (versionamento, hospedagem)	Todos		Sim	
Construir tabuleiro com peças	Marcelo Alfonso		Sim	
Implementar seleção de peça	Marcelo Alfonso		Sim	
Implementar movimentação de peça	Marcelo Alfonso		Sim	
Implementar remoção de peça	Marcelo Alfonso		Sim	
Fazer validação dos movimentos	Victor Oliveira		Sim	
Construir botão para reiniciar o jogo	Marcelo Alfonso		Sim	
Desenvolver exibição de feedback para o usuário	Victor Oliveira		Sim	
Executar testes	Daniel Izumi		Sim	
Aplicar correções	Daniel Izumi		Sim	Além das correções das falhas identificadas nos testes, executou-se algumas melhorias na interface do jogo.
Fazer deploy da aplicação	Todos		Sim	
Monitorar desempenho da aplicação	Todos		Sim	
Garantir documentação suficiente	Daniel Izumi		Sim	
Revisar e ajustar pontos do projeto	Anderson Akira		Sim	
Preparar apresentação final	Anderson Akira		Sim	

- Cada integrante contribuiu com 25% do desenvolvimento, considerando não somente a quantidade e as horas das tarefas, mas também a complexidade destas.

VERSÃO FINAL DO PRODUTO



Tela do jogo



Mensagem de vitória

CONCLUSÃO



- A maior dificuldade foi testar as funções que englobam elementos do navegador como *localStorage*, *navigator*, entre outros. A solução foi implementar mocks;
- A divisão do projeto em sprints permitiu um desenvolvimento incremental e focado, garantindo entregas constantes e acompanhamento preciso do progresso;
- A integração entre os módulos de tabuleiro, movimentação e controle de estado mostrou-se desafiadora, mas foi crucial para criar uma experiência de jogo fluida e consistente.

**OBRIGADO PELA
ATENÇÃO**